

- Panduan Bagian Kode untuk Laporan Skripsi
 - Bagian 1: Inisialisasi Rute Awal (Solomon's Sequential Insertion)
 - Potongan Kode:
 - 🕒Bagian 2: Penyisipan Dinamis & Aturan Non-Preemption (Cheapest Insertion)
 - Potongan Kode:
 - Bagian 3: Pencarian Lokal (Randomized Variable Neighborhood Descent - RVND)
 - Potongan Kode:
 - Bagian 4: Metaheuristik Iterated Local Search (ILS) & Perturbasi
 - Potongan Kode:

Panduan Bagian Kode untuk Laporan Skripsi

Dokumen ini memuat **4 bagian kode program utama** dari file **[solver.py]** (**file:///d:/PORTFOLIO/NUR/NUHA/solver.py**) yang sangat penting secara teoritis dan teknis untuk dimasukkan ke dalam laporan skripsi (baik di Bab IV Pembahasan maupun bagian Lampiran Kode). Setiap kode dilengkapi dengan penjelasan fungsi akademisnya.

Bagian 1: Inisialisasi Rute Awal (Solomon's Sequential Insertion)

- **Lokasi File:** `solver.py` (Baris 185 - 262)
- **Fungsi Akademis:** Membangun solusi rute awal dari kumpulan pelanggan statis (pesanan yang sudah masuk sebelum operasional berjalan). Algoritma mencari pelanggan terjauh dari depot sebagai titik awal (*seed*), lalu menyisipkan pelanggan lain secara bertahap pada posisi dengan biaya tambahan jarak terkecil (*cheapest cost*).

Potongan Kode:

```

def sequential_insertion(customer_ids: List[int],
                        dist_mtx: np.ndarray,
                        time_mtx: np.ndarray,
                        customers: Dict[int, Customer],
                        depot: Depot) -> List[Route]:
    unvisited = set(customer_ids)
    routes = []
    tw_limit = depot.tw_close - depot.tw_open

    while unvisited and len(routes) < depot.max_vehicles:
        # Pilih seed customer: pelanggan terjauh dari depot yang belum dikunjungi
        feasible_seeds = [c for c in unvisited if customers[c].demand <=
depot.capacity]
        if not feasible_seeds:
            break

        seed = max(feasible_seeds, key=lambda c: dist_mtx[0][c])

        # Uji kelayakan awal untuk rute [0, seed, 0]
        t_check = time_mtx[0][seed] + customers[seed].service_time + time_mtx[seed]
[0]

        if t_check > tw_limit:
            unvisited.discard(seed)
            continue

        # Inisialisasi rute dengan seed
        path = [0, seed, 0]
        unvisited.discard(seed)

        # Sisipkan pelanggan lain secara sekuensial ke rute saat ini
        while True:
            best_cost = float("inf")
            best_cust = -1
            best_pos = -1

            for c in unvisited:
                # Coba sisipkan c ke setiap posisi yang memungkinkan
                for pos in range(1, len(path)):
                    candidate_path = path[:pos] + [c] + path[pos:]

                    # Cek kelayakan (kapasitas & waktu)
                    if is_route_feasible(candidate_path, time_mtx, customers,
depot):
                        i_node = path[pos-1]
                        j_node = path[pos]
                        cost = dist_mtx[i_node][c] + dist_mtx[c][j_node] -
dist_mtx[i_node][j_node]

                        if cost < best_cost:
                            best_cost = cost
                            best_cust = c
                            best_pos = pos

            # Jika ditemukan pelanggan yang layak, lakukan penyisipan
            if best_cust != -1:
                path.insert(best_pos, best_cust)

```

```

        unvisited.discard(best_cust)
    else:
        break

    r = Route(path=path)
    r = update_route_metrics(r, dist_mtx, time_mtx, customers)
    routes.append(r)

return routes

```

🕒 Bagian 2: Penyisipan Dinamis & Aturan Non-Preemption (Cheapest Insertion)

- **Lokasi File:** `solver.py` (Baris 269 - 340)
- **Fungsi Akademis:** Algoritma dinamis untuk menangani pesanan mendadak (pelanggan dinamis) yang muncul pada `current_time`. Algoritma menghitung batas titik mana saja yang sudah dikunjungi atau dilewati kendaraan (*committed boundary*), lalu menyisipkan pesanan baru di sisa jalur yang belum ditempuh agar kendaraan tidak perlu berbalik arah (*non-preemption*).

Potongan Kode:

```

def cheapest_insertion(new_customer_id: int,
                      routes: List[Route],
                      dist_mtx: np.ndarray,
                      time_mtx: np.ndarray,
                      customers: Dict[int, Customer],
                      depot: Depot,
                      current_time: float) -> List[Route]:
    best_cost = float("inf")
    best_route_idx = -1
    best_pos = -1

    for r_idx, route in enumerate(routes):
        old_dist = route.total_distance

        # Hitung waktu kedatangan di setiap node sepanjang rute ini
        arrival_times = get_route_arrival_times(
            route.path, time_mtx, customers, depot.tw_open
        )

```

```

        # Cari batas indeks node yang sudah committed (dilalui sebelum
current_time)
        committed_boundary = 0
        for idx, arr_t in enumerate(arrival_times):
            if arr_t <= current_time:
                committed_boundary = idx

        # Posisi penyisipan hanya setelah committed_boundary (non-preemption)
        for pos in range(committed_boundary + 1, len(route.path)):
            new_path = route.path[:pos] + [new_customer_id] + route.path[pos:]

            # Cek kelayakan rute baru (kapasitas + waktu)
            if not is_route_feasible(new_path, time_mtx, customers, depot):
                continue

            new_dist = route_distance(new_path, dist_mtx)
            cost_increase = new_dist - old_dist

            if cost_increase < best_cost:
                best_cost = cost_increase
                best_route_idx = r_idx
                best_pos = pos

        if best_route_idx >= 0:
            routes[best_route_idx].path.insert(best_pos, new_customer_id)
            routes[best_route_idx] = update_route_metrics(
                routes[best_route_idx], dist_mtx, time_mtx, customers
            )
        else:
            # Jika tidak muat di rute mana pun, buat rute baru (jika kendaraan masih
ada)
            if len(routes) < depot.max_vehicles:
                new_path = [0, new_customer_id, 0]
                if is_route_feasible(new_path, time_mtx, customers, depot):
                    r = Route(path=new_path)
                    r = update_route_metrics(r, dist_mtx, time_mtx, customers)
                    routes.append(r)

    return routes

```

Bagian 3: Pencarian Lokal (Randomized Variable Neighborhood Descent - RVND)

- **Lokasi File:** `solver.py` (Baris 602 - 669)
- **Fungsi Akademis:** Mesin optimasi lokal untuk memperbaiki rute. Algoritma mengevaluasi 5 jenis struktur ketetangan (*neighborhood structures*) secara acak: 3 operator di dalam rute (2-Opt, Or-Opt, Exchange) dan 2 operator antar rute

(Relocate, Swap). Perbaikan dilakukan berulang kali sampai tidak ada perbaikan jarak yang bisa dicapai di semua tetangga (*local optimum*).

Potongan Kode:

```
def rvnd_optimize(routes: List[Route],
                  dist_mtx: np.ndarray,
                  time_mtx: np.ndarray,
                  customers: Dict[int, Customer],
                  depot: Depot,
                  current_time: float = 0.0) -> Tuple[List[Route], List[Dict]]:
    routes = copy.deepcopy(routes)
    log = []

    # Operator intra-rute (dalam rute yang sama)
    intra_ops = [
        ("2-Opt", two_opt_improve),
        ("Or-Opt", or_opt_improve),
        ("Exchange", exchange_improve),
    ]
    for r_idx in range(len(routes)):
        for name, op in intra_ops:
            routes[r_idx], imp = op(
                routes[r_idx], dist_mtx, time_mtx, customers, depot, current_time
            )

    # Operator inter-rute (antar dua rute berbeda)
    inter_ops = [
        ("Relocate", relocate_inter),
        ("Swap(1,1)", swap_inter),
    ]

    improved = True
    while improved:
        improved = False
        ops = inter_ops[:]
        random.shuffle(ops)

        for name, op in ops:
            routes, imp = op(routes, dist_mtx, time_mtx, customers, depot,
                             current_time)
            if imp:
                log.append({"operator": name, "type": "inter", "status":
                    "diperbaiki"})
                improved = True

    # Optimasi ulang bagian intra-rute setelah struktur inter berubah
    for r_idx in range(len(routes)):
        for iname, iop in intra_ops:
            routes[r_idx], iimp = iop(
                routes[r_idx], dist_mtx, time_mtx,
                customers, depot, current_time
            )
```

```
break
```

```
return routes, log
```

Bagian 4: Metaheuristik Iterated Local Search (ILS) & Perturbasi

- **Lokasi File:** `solver.py` (Baris 805 - 864)
- **Fungsi Akademis:** Algoritma kendali tingkat tinggi (*metaheuristic*) untuk mencegah solusi terjebak pada optimum lokal (*local optima*). ILS melakukan pencarian lokal intensif, kemudian secara sengaja merusak/mengacak rute secara layak (*perturbation*), dan mengoptimalkannya kembali untuk menemukan rute yang jauh lebih pendek.

Potongan Kode:

```
def iterated_local_search(routes: List[Route],
                          dist_mtx: np.ndarray,
                          time_mtx: np.ndarray,
                          customers: Dict[int, Customer],
                          depot: Depot,
                          current_time: float = 0.0,
                          max_iters: int = 15,
                          random_seed: int = None) -> Tuple[List[Route],
List[Dict]]:
    if random_seed is not None:
        random.seed(random_seed)
        np.random.seed(random_seed)

    log = []

    # Fase 1: Optimasi awal (Local Search berturut-turut + RVND)
    curr_routes = copy.deepcopy(routes)
    curr_routes = local_search_optimize(curr_routes, dist_mtx, time_mtx, customers,
depot, current_time)
    curr_routes, rvnd_log = rvnd_optimize(curr_routes, dist_mtx, time_mtx,
customers, depot, current_time)
    log.extend(rvnd_log)

    best_routes = copy.deepcopy(curr_routes)
    best_dist = total_distance(best_routes, dist_mtx)

    # Fase 2: Iterasi ILS dengan Perturbasi
    for i in range(max_iters):
```

```

# a. Terapkan Perturbasi (Pengacakan acak yang layak)
perturbed = perturbation(best_routes, dist_mtx, time_mtx, customers, depot,
current_time)
    if total_distance(perturbed, dist_mtx) == best_dist and len(perturbed) ==
len(best_routes):
        continue

# b. Local Search pada rute terperturbasi
perturbed_ls = local_search_optimize(perturbed, dist_mtx, time_mtx,
customers, depot, current_time)

# c. RVND pada hasil local search
perturbed_rvnd, p_log = rvnd_optimize(perturbed_ls, dist_mtx, time_mtx,
customers, depot, current_time)

# d. Kriteria Penerimaan (Acceptance Criterion)
new_dist = total_distance(perturbed_rvnd, dist_mtx)
if new_dist + 1e-9 < best_dist:
    best_dist = new_dist
    best_routes = copy.deepcopy(perturbed_rvnd)
    log.append({
        "operator": f"Perturbasi & ILS (Iterasi {i+1})",
        "type": "metaheuristic",
        "status": f"diperbaiki (jarak berkurang menjadi {best_dist:.2f}
km)"
    })
    log.extend(p_log)

return best_routes, log

```